TokenFlow: Distributed LLM Serving and Load Balancing

CHEONG KIN NGAI (Bankey) · CISC3010 · Topic 14

1. Question and system

When requests with different prompt and output lengths share an inference service, a routing decision can affect where they wait. This project asks how round-robin and least-outstanding routing affect throughput, time to first token (TTFT), and tail completion time. It also examines iteration-level scheduling, shared computation, and worker failure. TokenFlow wraps the pretrained Qwen2.5-0.5B-Instruct model [2]; it does not train a model.

The deployment has a client, an HTTP gateway process, and two independent worker processes on one Windows host. The gateway admits requests, selects a healthy worker, relays streamed token events, and records assignment and final status. Each worker holds a complete model replica, a FIFO waiting queue, an active set, and per-request KV state. The client and process traces record arrivals, worker assignments, first tokens, completion, and output lengths. Worker slots and KV reservations bound active work. An independent checker examines saved traces and cleanup state.

```
Client -> Gateway -> Worker 1: queue -> scheduler -> Qwen
                  -> Worker 2: queue -> scheduler -> Qwen
          request and worker traces -> checker / metrics
```

Round robin rotates among healthy workers. Least outstanding chooses the worker with the fewest assigned requests that have not terminated, including queued and running requests. Selection and count updates are protected by one gateway lock. A long and a short request each contribute one to this score. Gateway admission additionally enforces local client quotas and returns HTTP 429 with a retry hint when a limit is exceeded.

2. Reading and implementation

Orca [1] schedules generation at iteration boundaries and uses selective batching. TokenFlow adapts these ideas inside each worker: completed requests release slots, and waiting requests may join a later model step. Real-model selective execution combines request-token pieces for projections, normalization and MLP while Attention and KV state remain request-local. Decode tokens can run alongside bounded prefill chunks. A shared 128-token prefill allowance limits the prompt work in one worker step, not its wall-clock time.

Routing is a separate decision made by the gateway for the entire request. Orca's model-parallel GPU engine is not reproduced: this project uses full CPU model replicas and PyTorch/Transformers components [3, 4]. The simulator uses explicit prefill/decode costs and deterministic outputs. Its timing results are not real-model speedup claims.

3. Required routing comparison

The primary study uses real Qwen FP32 inference on an Intel Core Ultra 5 338H, with 12 logical CPUs and about 31.5 GiB RAM. Two workers share the host; each has two model threads and four active slots. Each trial replays the same 120 short/long requests with fixed output limits. Arrivals are open loop at fixed intervals: 1/s or 3/s. Workers warm up before measurement. At each rate, each routing policy runs six times, with policy order alternated and the same workload and resource settings within each pair. This gives 24 trials and 2,880 requests.

Arrivals/s	Policy	Requests/s	P95 TTFT (s)	P95 completion (s)
1	Round robin	0.964 ± 0.017	3.72 ± 1.29	13.13 ± 1.85
1	Least outstanding	0.971 ± 0.005	2.37 ± 0.29	11.53 ± 1.09
3	Round robin	1.294 ± 0.061	45.44 ± 4.40	52.63 ± 4.76
3	Least outstanding	1.269 ± 0.057	45.69 ± 2.75	53.34 ± 3.73

Table 1. Mean ± sample standard deviation across six runs. P95 is calculated within each run; it is not a percentile of pooled requests. Throughput is completed requests divided by the interval from first scheduled dispatch to final client outcome. Client-observed TTFT and completion time use the same monotonic clock.

At 1/s, mean trial P95 TTFT was lower with least outstanding. The paired mean reduction was 1.35 s, with an approximate 95% interval of 0.23 to 2.48 s. The corresponding throughput intervals at both rates include zero, so the data do not establish a reliable throughput winner or equivalence. Request count is an imperfect proxy for remaining work, and round robin already distributed this fixed mix reasonably evenly; these are plausible explanations, not proven causes.

At 3/s, mean worker full-slot fractions were 92.6% and 90.3% for round robin and least outstanding. The router used about 10.5% and 10.7% of one logical core. This supports worker-side queueing pressure and no observed router CPU overload. Slot occupancy does not establish that all CPU cores or memory bandwidth were saturated. A separate intentionally alternating arrival-order diagnostic adds 16 trials and 1,024 requests; it tests a constructed stress case, not representative production traffic.

4. Scheduling, execution, and failure

Admission in the simulator. A separate 12-trial experiment compared static cohorts with continuous admission at 6/s and 18/s, with 48 requests per trial, three repeats per setting, the same workload and cost model, least-outstanding routing, and four slots per worker. At 6/s the mean trial P95 TTFT was 6.392 s for static versus 1.893 s for continuous; at 18/s it was 10.941 versus 5.433 s. A new 12-trial rerun on 28 October showed the same direction: 7.36 versus 1.29 s at 6/s and 11.85 versus 5.12 s at 18/s. The simulator models decode step time as (0.012 + 0.002 × active decode requests) seconds and prefill chunk time as (0.003 + 0.00025 × chunk tokens) seconds. These numbers describe that model, rather than Qwen inference.

Selective execution on real Qwen. In a separate 24-request workload with three repeats per setting, both settings used continuous admission and the same bounded prefill. Changing independent model calls to selective execution raised mean throughput from 0.787 to 1.154 requests/s, or 46.7%; mean trial P95 TTFT fell from 19.18 to 11.98 s. The comparison is confined to this CPU workload and execution configuration; it is not a routing result.

Chunked prefill on real Qwen. With selective execution held fixed, the 128-token allowance changed mean trial P95 inter-token gaps from 544.6 to 854.6 ms, while P99 fell from 1,962.9 to 913.7 ms. Long pauses were reduced while some shorter-tail gaps worsened. Three repeats were collected; P99 was examined after the initial analysis, so a larger prospective test is needed before generalizing.

Worker failure and cancellation. In one real-model fault run, stopping a worker during 16 requests yielded nine completions and seven explicit failures. Detection took 0.554 s in that run, and new requests used the surviving worker. The stopped process was restarted manually. An interrupted partial answer is not resumed or silently replayed. In a separate 12-request cancellation run, nine requests completed and three were cancelled; the checker found no remaining reservations.

5. Correctness, limits, and submission

Twelve real-model reference cases matched token IDs from Transformers generation. An FP32 numerical audit also checked logits and KV values within recorded tolerances. The final routing archive contains all 40 real-model trials, 3,904 requests and 108,800 output tokens. Reanalysis checks workload identity, ordered tokens, terminal outcomes, model-step accounting, cleanup and matching greedy token IDs within each workload family. On 28 October, 32 automated tests passed. These results validate the observed cases, not every input or failure mode.

The workload uses repeated synthetic short/long prompt templates and two fixed arrival rates; it is not a representative user trace or an answer-quality benchmark. All workers share one host and its CPU and memory. The gateway and host are single failure points. The prototype does not implement multi-host orchestration, model sharding, PagedAttention, real cross-request prefix KV reuse, or seamless replay of interrupted output. Fixed-interval arrivals, limited repetitions, shared-host interference and post-hoc tail analysis constrain generalization.

No specific feedback was received after the presentation. Following the presentation, the code and deliverables were reviewed against the submission checklist, raw evidence and a portable verifier were prepared, and the simulator scheduling comparison and automated checks were rerun for submission. The repository contains setup commands, the demo recording, workloads and traces, a checker, plotting scripts and the report. Raw model weights are downloaded separately.

Implementation and AI use. The project adds the gateway, worker management, request lifecycle, scheduling, selective execution, observability, correctness checks and experiments around pretrained Qwen and PyTorch/Transformers. Codex provided substantial assistance with design, code, tests, experiments and documentation; the use and validation are disclosed in `AI_USE.md`. External components and papers are attributed below.

References

[1] G.-I. Yu et al. *Orca: A Distributed Serving System for Transformer-Based Generative Models*. OSDI, 2022. https://www.usenix.org/conference/osdi22/presentation/yu

[2] Qwen Team. *Qwen2.5-0.5B-Instruct model card*. https://huggingface.co/Qwen/Qwen2.5-0.5B-Instruct

[3] PyTorch. *PyTorch documentation*. https://pytorch.org/docs/

[4] Hugging Face. *Transformers documentation*. https://huggingface.co/docs/transformers/

[5] A. Agrawal et al. *Sarathi-Serve: Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve*. OSDI, 2024. https://www.usenix.org/conference/osdi24/presentation/agrawal

[6] Y. Zhong et al. *DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving*. OSDI, 2024. https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin